

Technical Summary:

GoT: Effective Graph-of-Thought Reasoning in Language Models

Kai-Yu Lu

1 Research Problem and Motivation

Reasoning with Language Models (LMs) has been substantially improved by Chain-of-Thought prompting (CoT), which generates intermediate natural-language rationales before producing a final answer. However, human reasoning is not restricted to sequential chains. Human cognition frequently forms non-linear connections between ideas, enabling deductive leaps that are difficult to express as a single linear sequence. Prior CoT-style approaches, including multimodal variants, primarily treat reasoning as a chain and therefore omit explicit structural information among thought units. This gap motivates Graph-of-Thought reasoning (GoT), which models thought units as nodes and their connections as edges, with the goal of enabling LMs to exploit non-sequential structure for more realistic and accurate reasoning across text-only and multimodal settings.

2 Related Work

CoT methods can be categorized into prompting-based and fine-tuning-based approaches. Zero-shot and few-shot CoT prompting improves reasoning by eliciting intermediate steps, but it does not explicitly encode structural relations among thought units. Auto-CoT reduces manual demonstration design by automatically generating reasoning demonstrations, but reasoning remains chain-based. Tree-of-Thoughts (ToT) explores multiple coherent thought units via tree-structured search and evaluation, but its structure is restricted to trees rather than graphs with cross-branch links. Multimodal-CoT integrates visual information into a two-stage CoT framework via feature fusion, but it still lacks an explicit deductive structure among thought units. GoT differs from ToT by augmenting a two-stage rationale-then-answer pipeline with an explicit graph representation intended to capture non-sequential connections, and differs from Multimodal-CoT by adding a dedicated thought-graph encoder and graph-aware integration beyond text and vision features.

3 Dataset Construction

No new dataset is constructed. Experiments use two established benchmarks, AQUA-RAT and ScienceQA, which provide supervised training data for rationale generation and answer prediction.

Datasets and Statistics

Table 1: Datasets, sources, and splits. Missing items are reported as “Not specified in the paper.”

Dataset	Description and Source	Train	Dev	Test
AQUA-RAT	Algebraic word problems with natural-language rationales; sourced from Ling et al. (2017).	97,467	254	254
ScienceQA	Large-scale multimodal science QA with detailed lectures and explanations; sourced from Lu et al. (2022).	12,726	4,241	4,241

Table 2: ScienceQA dataset attributes.

Attribute	Value
#Total questions	21,208
#Subjects	3
#Topics	26
#Categories	127
#Skills	379

Preprocessing and Annotation

AQUA-RAT and ScienceQA are used as provided by their original sources. For multimodal GoT, an image caption is appended to the input text to incorporate image information into thought-graph construction. Coreference resolution and open information extraction are applied to input text to construct thought graphs. Additional dataset preprocessing details beyond these steps are not specified in the paper.

4 Query Protocol and Task Definitions

Task Type

Both benchmarks are treated as multiple-choice reasoning tasks with a two-stage generation protocol:

- **Stage 1, Rationale Generation:** generate intermediate rationales conditioned on the input.
- **Stage 2, Answer Generation:** generate the final answer conditioned on the original input concatenated with the predicted rationales from Stage 1.

Input and Output Format

- **Text-only setting (AQUA-RAT):** input text is the concatenation of question, context, and choices. The output is a generated rationale (Stage 1) and a final answer choice (Stage 2).
- **Multimodal setting (ScienceQA):** input text additionally appends an image caption. Image features are encoded by a vision encoder, and thought-graph features are constructed from the text (including the caption). Outputs follow the same two-stage protocol.

Success Criteria

- **Primary metric:** Accuracy (ACC) of the final predicted answer on the test set.
- **Rationale quality:** ROUGE-L is reported for rationale generation. Additional metrics for rationale generation quality are reported in an appendix, including BLEU-1, BLEU-4, ROUGE, and cosine similarity over sentence embeddings.

Not specified in the paper: any explicit constraint on rationale format beyond being natural language, any special decoding constraints, and any human evaluation protocol for rationales.

5 Modeling Approach

Overview

GoT augments a two-stage encoder-decoder reasoning framework with a Graph-of-Thought representation and encoder. The system includes (i) thought-graph construction via Extract-Clustering-Coreference (ECC), (ii) separate encoders for text, vision (optional), and thought graph, (iii) cross-attention alignment between text tokens and image patches or graph nodes, and (iv) a gated fusion mechanism that integrates modalities before decoding rationales and answers.

Base Text Representation

$$H_T = \{h_0, h_1, \dots, h_l\} = \text{Encoder}_{\text{text}}(S) \quad (1)$$

Purpose. Define the token-level text representation used as the backbone input to cross-attention and fusion, for both rationale generation and answer generation stages.

Symbols. $S = \{w_0, \dots, w_l\}$ is the input token sequence, l is the sequence length, $\text{Encoder}_{\text{text}}(\cdot)$ is a Transformer encoder (such as T5), h_i is the hidden state of token i , and H_T is the set of hidden states from the last encoder layer.

Intuition. Each token is mapped to a continuous vector that summarizes its meaning given the full input context, enabling subsequent modules to attend to tokens and combine information across modalities.

Usage. H_T is computed for the input text in Stage 1, and for the concatenation of input text and predicted rationales in Stage 2. H_T serves as the query representation in cross-attention alignment with image and graph features, and as the primary stream in gated fusion.

GoT Construction via Extract-Clustering-Coreference (ECC)

ECC constructs a thought graph $G(N, E)$ from the input text, where N is a set of thought-unit nodes and E is an adjacency matrix indicating edges. ECC (i) extracts subject-verb-object triplets using an Open Information Extraction system, and (ii) performs coreference resolution using Stanford CoreNLP to cluster mentions that refer to the same entity, replacing nodes in each cluster with a representative mention, then reconnecting edges accordingly. This procedure is designed to yield denser graphs that better support deductive reasoning patterns of the form $x \rightarrow y$ and $y \rightarrow z$ implying $x \rightarrow z$.

Node Embedding for Thought Units

$$p = [\langle s \rangle, n_0, \langle /s \rangle, \dots, \langle s \rangle, n_j, \langle /s \rangle] \quad (2)$$

Purpose. Define how each thought-unit node is encoded into an initial vector representation using the same text encoder as the input stream.

Symbols. $N = \{n_0, \dots, n_j\}$ is the set of graph nodes (thought units), p is the constructed node-input sequence, and $\langle s \rangle$ and $\langle /s \rangle$ are special boundary tokens used to highlight each node span.

Intuition. By marking node boundaries, the encoder can produce a dedicated representation per node. The representation of $\langle s \rangle$ corresponding to each node span serves as a compact embedding of that thought unit.

Usage. The sequence p is fed into $\text{Encoder}_{\text{text}}(\cdot)$ to obtain initial node embeddings, which are then passed to a Graph Attention Network (GAT) to encode graph structure.

GoT Encoder and Feature Integration

The thought graph is encoded by a GAT, which computes node representations by attending over graph neighbors, producing a graph representation H_G . In multimodal settings, image patch features are extracted by a vision encoder and projected into the same shape as text features to obtain H_I . Cross-attention is applied to align text tokens with (i) image patches and (ii) graph nodes, producing attended image and graph features. A gated fusion layer then combines the aligned features with the text features.

$$\lambda = \begin{cases} \sigma(W_T H_T + W_G H_G) & \text{text-only} \\ \sigma(W_T H_T + W_I H_I + W_G H_G) & \text{multimodal} \end{cases} \quad H = \begin{cases} (1 - \lambda) \cdot H_T + \lambda \cdot H_G & \text{text-only} \\ (1 - \lambda) \cdot H_T + \lambda \cdot H_I + \lambda \cdot H_G & \text{multimodal} \end{cases} \quad (3)$$

Purpose. Compute a fused representation H that adaptively integrates text, graph, and image information prior to decoding rationales or answers.

Symbols. $\sigma(\cdot)$ is the sigmoid function producing values in $(0, 1)$, λ is a gating weight, H_T is the text representation, H_G is the thought-graph representation, H_I is the vision representation (multimodal only), and W_T, W_I, W_G are trainable weights.

Intuition. The gate λ controls how much non-text information is injected. When λ is larger, more weight is placed on thought-graph and image features; when smaller, the model relies more on text.

Usage. The fused feature H is fed to a Transformer decoder to generate Stage 1 rationales and Stage 2 final answers. In Stage 2, H_T is recomputed from the concatenated input that includes the predicted rationales, and the thought graph is reconstructed accordingly.

Minimal Walkthrough Example

Given an input that contains “Earthquake comes from earth and quake” and “earth means ground” and “quake means shake”, ECC extracts triplets linking {Earthquake, comes from, earth}, {Earthquake, comes from, quake}, {earth, means, ground}, and {quake, means, shake}, then clusters coreferent mentions if present. The resulting thought graph explicitly connects {Earthquake} to {earth, quake} and further to {ground, shake}. The GAT encodes these connections into H_G , which is fused with H_T so that the decoder can more directly leverage the non-linear connection from Earthquake to ground and shake when generating the rationale and selecting the final answer.

6 Empirical Results

Experimental Setup

Table 3: Training, architecture, and environment settings. Missing items are reported as “Not specified in the paper.”

Item	Setting
Backbone model	T5-base and T5-large, initialized from FLAN-Alpaca checkpoints for fair comparison
Vision encoder	ViT-large in the main multimodal setting; DETR in an alternative configuration study
Training epochs	100
Learning rate	5×10^{-5}
Batch size (per device)	T5-base: 10; T5-large: 8
Weight decay	0.01
Max input length	512
Max number of nodes	150
Hardware	Four NVIDIA A800 80G GPUs
Optimizer, decoding details	Not specified in the paper.

Main Results on AQUA-RAT (Text-only)

Table 4: AQUA-RAT test accuracy (ACC, %) for key fine-tuned baselines and GoT.

Model	Size	ACC (%)
FLAN-Alpaca (CoT fine-tuning)	223M	30.09 ± 1.12
GoT-T5	223M	32.09 ± 1.62
FLAN-Alpaca (CoT fine-tuning)	738M	33.73 ± 1.14
GoT-T5	738M	34.48 ± 1.11

GoT improves answer accuracy from 30.09 to 32.09 for the 223M setting, a 2.00 percentage point gain, and from 33.73 to 34.48 for the 738M setting, a 0.75 percentage point gain. The paper reports a 0.78 increase in ROUGE-L for Stage 1 rationale generation for the base model, and attributes limited rationale gains to restricted information used in thought-graph construction in the text-only setup.

Main Results on ScienceQA (Multimodal)

Table 5: ScienceQA test accuracy (AVG, %) for Multimodal-CoT and GoT with matched backbone sizes.

Model	Size	AVG ACC (%)
Multimodal-CoT	223M	85.19
GoT-T5	223M	87.59 ± 0.20
Multimodal-CoT	738M	90.45
GoT-T5	738M	90.81 ± 0.12
Human	Not applicable	88.40

With the T5-base backbone, GoT boosts ScienceQA accuracy from 85.19 to 87.59, a 2.40 percentage point improvement over Multimodal-CoT under the same backbone size. With T5-large, GoT reaches 90.81 versus 90.45 for Multimodal-CoT, a smaller but consistent gain.

Ablation Studies

Table 6: AQUA-RAT ablations for GoT-T5 base (ACC, %).

Variant	ACC (%)	Δ vs. GoT
GoT-T5 (base)	32.09	0.00
Random Thought Graph	30.31	-1.78
Triplets Concatenation	31.20	-0.89
Coreference Injection	30.32	-1.77

AQUA-RAT.

Table 7: ScienceQA results under UnifiedQA + DETR settings (ACC, %). Missing items are reported as “Not specified in the paper.”

Model	ACC (%)	Δ
Multimodal-CoT (base)	77.67	Not specified in the paper.
GoT (base)	81.21	+3.54 over Multimodal-CoT (base)
Multimodal-CoT (base, enlarged gated fusion)	79.17	-2.04 vs. GoT (base)
GoT (base) with Random Thought Graph	76.74	-4.47 vs. GoT (base)

ScienceQA configuration study and ablations.

Efficiency and Cost Discussion

Table 8: Reported parameters and inference throughput (eval samples per second).

Model	#Parameters	Throughput
Multimodal-CoT (base)	227M	16.33
GoT (base)	233M	13.38

GoT increases the reported parameter count from 227M to 233M in the base multimodal setting and reduces inference throughput from 16.33 to 13.38 eval samples per second, reflecting additional computation from thought-graph construction and encoding.

7 Summary

Graph-of-Thought reasoning introduces an explicit thought-graph representation within a two-stage rationale-then-answer framework. An ECC construction pipeline transforms input text into a thought graph via OpenIE triplets and coreference-based clustering, and a dedicated GAT encoder produces graph features that are fused with text and, when available, vision features through gated fusion. Empirical results show accuracy improvements on AQUA-RAT from 30.09 to 32.09 (T5-base) and on ScienceQA from 85.19 to 87.59 (T5-base), with ablations demonstrating that structured graphs drive the gains. Reported limitations include increased computational cost and reduced throughput, as well as constrained improvements in rationale generation when graph construction is based on limited input text.